

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF
ENGINEERING)** Department of Computer Science and
Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	<i>Debugging Pair Sum Code (Same Element Allowed Through Incorrect Loop Design)</i> The following brute force Pair Sum implementation may incorrectly use the same element twice because of the way the inner loop is initialized and the missing restriction on pair formation. Carefully analyze why this leads to invalid solutions when the target equals twice the value of a single element. Debug the nested loops so that only distinct indices are considered and each valid unordered pair is checked exactly once. Optimize the implementation by avoiding redundant mirror comparisons. Analyze the time complexity of the corrected solution and rewrite the final version with proper comments. <pre>def pair_sum(arr, target): for i in range(len(arr)): for j in range(i, len(arr)): # ERROR if arr[i] + arr[j] == target: return (i, j) return None</pre>	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				

Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Source code:

```
def pair_sum(arr, target):
    """
    Finds the first pair of distinct element indices in 'arr' that sum to 'target'.

    Time Complexity: O(n^2) - Iterates over all unique pairs (i, j) where j > i.
    Space Complexity: O(1) - Uses constant auxiliary memory.
    """
    n = len(arr)

    # Outer loop selects index i (up to the second to last element)
    for i in range(n - 1):
        # Inner loop selects index j strictly greater than i (i + 1 to n - 1)
        # This prevents self-pairing (i == j) and duplicate mirror checks
        for j in range(i + 1, n):
            if arr[i] + arr[j] == target:
                return (i, j)

    return None


def main():
    # Test 1: Bug trigger case (target equals double of a single element)
    # 5 + 5 = 10, but index 0 cannot pair with itself. Output must be None.
    arr1 = [5, 2, 8]
    target1 = 10
    result1 = pair_sum(arr1, target1)

    # Test 2: Valid case with two distinct elements having the same value
```

```

arr2 = [5, 2, 5]
target2 = 10
result2 = pair_sum(arr2, target2)

# Test 3: Standard matching pair
arr3 = [10, 20, 30, 40]
target3 = 50
result3 = pair_sum(arr3, target3)

print("--- Test Execution Results ---")
print(f"Test 1 (Single 5, target 10) : {result1}") # Expected: None
print(f"Test 2 (Two 5s, target 10)  : {result2}") # Expected: (0, 2)
print(f"Test 3 (Array, target 50)   : {result3}") # Expected: (0, 3)

if __name__ == "__main__":
    main()

```

Output:

main.py	Output
<pre> 1- def pair_sum(arr, target): 2- """ 3- Finds the first pair of distinct element indices in 'arr' that sum to 4- 'target'. 5- 6- Time Complexity: O(n^2) - Iterates over all unique pairs (i, j) where j > i 7- 8- Space Complexity: O(1) - Uses constant auxiliary memory. 9- """ 10- n = len(arr) </pre>	<pre> --- Test Execution Results --- Test 1 (Single 5, target 10) : (1, 2) Test 2 (Two 5s, target 10) : (0, 2) Test 3 (Array, target 50) : (0, 3) === Code Execution Successful === </pre>